Volume 7 – LangChain4j Reference Implementation

Version: 1.0

Purpose

This document describes the practical implementation of the Enterprise Tiny Agent Framework using LangChain4j.

The goal is to leverage LangChain4j for:

- LLM Integration
- Tool Calling
- Chat Memory
- AI Services
- Prompt Management
- Structured Output

while preserving the architectural principles defined in Volumes 1–6.

This implementation is intended for:

```
Java 21+
Spring Boot 3.x
LangChain4j
DuckDB
Apache Calcite
PostgreSQL
Ollama
OpenAI Compatible APIs
```

Architecture Mapping

Framework Concept	LangChain4j Component
Agent	AI Service
Tool	@Tool Method

Framework Concept	LangChain4j Component
Tool Registry	Spring Bean Registry
Model Provider	ChatLanguageModel
Memory Provider	ChatMemory
Agent Runtime	AI Service Runtime
Model Router	Custom Factory
Agent Definition	Spring Configuration
Agent Registry	Spring Context

Recommended Project Structure

```
metadata-ai-platform

├── platform-common
│
├── platform-models
│
├── platform-tools
│
├── platform-agents
│
├── platform-runtime
│
├── platform-memory
│
├── platform-api
│
└── platform-server
```

Maven Dependencies

Core LangChain4j

```
<dependency>
    <groupId>dev.langchain4j</groupId>
```

```
    <artifactId>langchain4j</artifactId>
</dependency>
```

Ollama

```
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-ollama</artifactId>
</dependency>
```

OpenAI Compatible APIs

```
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-open-ai</artifactId>
</dependency>
```

Spring Boot

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>
```

Model Configuration

Create a centralized model factory.

Model Router

```
public interface ModelRouter {

    ChatLanguageModel getModel(
```

```
            AgentType agentType);

}
```

Implementation

```
@Component
public class DefaultModelRouter
        implements ModelRouter {

    @Override
    public ChatLanguageModel getModel(
            AgentType agentType) {

        return switch (agentType) {

            case METADATA ->
                    metadataModel();

            case SQL ->
                    sqlModel();

            case RESEARCH ->
                    researchModel();

            default ->
                    metadataModel();
        };
    }
}
```

Agent Types

```
public enum AgentType {

    METADATA,

    SQL,

    RULE,
```

```
    DOCUMENTATION,

    RESEARCH,

    OPERATIONS,

    DATA_QUALITY

}
```

Metadata Tools

Metadata operations should always be implemented as tools.

Search Dataset Tool

```
@Component
public class SearchDatasetTool {

    @Tool("""
        Search metadata catalog.
        Returns matching datasets.
    """)
    public DatasetSearchResult search(
            String searchText) {

        return metadataRepository
                .search(searchText);
    }
}
```

Get Schema Tool

```
@Component
public class GetSchemaTool {

    @Tool("""
        Get schema for dataset.
    """)
```

```
    public DatasetSchema getSchema(
            String datasetName) {

        return metadataRepository
                .getSchema(datasetName);
    }
}
```

Get Lineage Tool

```
@Component
public class GetLineageTool {

    @Tool("""
        Retrieve dataset lineage.
    """)
    public DatasetLineage getLineage(
            String datasetName) {

        return metadataRepository
                .getLineage(datasetName);
    }
}
```

SQL Validation Tool

All generated SQL must pass through Calcite validation.

```
@Component
public class ValidateSqlTool {

    @Tool("""
        Validate SQL using Calcite.
    """)
    public ValidationResult validate(
            String sql) {

        return calciteService
                .validate(sql);
```

```
    }
}
```

DuckDB Query Tool

```
@Component
public class ExecuteQueryTool {

    @Tool("""
        Execute SQL against DuckDB.
    """)
    public QueryResult execute(
            String sql) {

        return duckDbService
                .execute(sql);
    }
}
```

Metadata Agent

The Metadata Agent is responsible for dataset discovery.

AI Service

```
public interface MetadataAgent {

    @SystemMessage("""
        You are a metadata specialist.

        Use tools.

        Never invent metadata.

        Return only discovered datasets.
    """)
    String findDataset(
```

```
            @UserMessage String request);
}
```

Factory

```
@Component
public class MetadataAgentFactory {

    public MetadataAgent create() {

        return AiServices.builder(
                    MetadataAgent.class)
                .chatLanguageModel(
                    modelRouter.getModel(
                        AgentType.METADATA))
                .tools(
                    searchDatasetTool,
                    getSchemaTool,
                    getLineageTool)
                .build();
    }
}
```

SQL Agent

Responsible for SQL generation only.

Never execute SQL.

AI Service

```
public interface SqlAgent {

    @SystemMessage("""
        Generate SQL only.

        Use schema tools.

        Validate before returning.
```

```
        Never execute SQL.
    """)
    String generateSql(
            @UserMessage String request);
}
```

Registered Tools

```
SearchDatasetTool

GetSchemaTool

ValidateSqlTool
```

Documentation Agent

Responsible for explaining metadata and platform behavior.

Tools

```
DocumentSearchTool

MetadataSearchTool

ArchitectureTool
```

Example Prompt

```
@SystemMessage("""
You explain platform behavior.

Always use retrieved documentation.
```

```
Never invent architecture details.
""")
```

Research Agent

Research is the only agent allowed to access external knowledge sources.

Tools

```
WebSearchTool

DocumentReaderTool

SummarizationTool
```

Model Recommendation

```
Qwen 14B

DeepSeek 14B

Hosted GPT
```

Structured Output Pattern

Avoid free-form text.

Use POJOs.

Example

```
public record DatasetMatch(

        String datasetName,
```

```
        String description,

        Double confidence

) {}
```

Response

```
public record DatasetSearchResponse(

        List<DatasetMatch> matches

) {}
```

Chat Memory

Most agents require only session memory.

Memory Provider

```
@Bean
public ChatMemory chatMemory() {

    return MessageWindowChatMemory
            .withMaxMessages(20);
}
```

Recommendation

Avoid:

```
Unlimited Memory
```

Use:

```
10-20 Messages
```

for most agents.

Agent Registry

Central location for agent discovery.

Interface

```
public interface AgentRegistry {

    Object getAgent(
            AgentType type);

}
```

Implementation

```
@Component
public class DefaultAgentRegistry
        implements AgentRegistry {

    private final Map<
        AgentType,
        Object> agents;

}
```

Observability

Capture every execution.

Execution Log

```
public record AgentExecutionLog(

        String requestId,

        String agentName,

        String modelName,

        Long durationMs,

        Integer toolCalls,

        Double confidence

) {}
```

Database Table

```
CREATE TABLE agent_execution_log (

    execution_id BIGSERIAL,

    request_id VARCHAR(100),

    agent_name VARCHAR(100),

    model_name VARCHAR(100),

    started_at TIMESTAMP,

    completed_at TIMESTAMP,

    duration_ms BIGINT,

    tool_calls INTEGER
);
```

Error Handling

Standardized error model.

Response

```
public record AgentError(

        String code,

        String message,

        String details

) {}
```

Error Types

```
VALIDATION_ERROR

TOOL_ERROR

MODEL_ERROR

SECURITY_ERROR

SYSTEM_ERROR
```

Security

Agents should never have unrestricted tool access.

Metadata Agent

Allowed:

```
SearchDatasetTool

GetSchemaTool

GetLineageTool
```

Forbidden:

```
RestartJobTool

DeleteDatasetTool
```

Configuration

Externalize everything.

YAML

```
agents:
  metadata:
    model: qwen3-4b
  sql:
    model: qwen3-coder-8b
  research:
    model: qwen3-14b
memory:
  max-messages: 20
```

REST API

Expose a generic execution endpoint.

Request

```
{
  "agentType": "SQL",
  "request": "Show top recharge amounts"
}
```

Response

```
{
  "success": true,
  "response": {
    "sql": "SELECT ..."
  }
}
```

Recommended Initial Agents

Phase 1:

```
Metadata Agent

SQL Agent

Documentation Agent
```

Phase 2:

```
Rule Agent

Data Quality Agent
```

Phase 3:

```
Research Agent

Operations Agent
```

Recommended Models

Metadata Agent

```
Qwen3 4B
```

SQL Agent

```
Qwen3-Coder 8B
```

or

```
DeepSeek-Coder 6.7B
```

Documentation Agent

```
Qwen3 4B
```

Research Agent

```
Qwen3 14B
```

Implementation Checklist

Before production:

- Model Router implemented
- Tool Registry implemented
- Agent Registry implemented
- Calcite Validation enabled
- DuckDB integration completed
- Structured outputs used
- Memory configured
- Logging enabled
- Security restrictions enabled
- Configuration externalized

Final Recommendation

LangChain4j should be used as the foundation of the Agent SDK.

Keep agents small and focused.

Place business logic inside tools.

Use Calcite as the SQL gatekeeper.

Use DuckDB strictly as an execution engine.

Treat Metadata Registry as the primary source of truth.

The combination of:

```
LangChain4j
+
Metadata Registry
+
Calcite
```

```
+
DuckDB
```

provides a lightweight, enterprise-grade AI platform that remains understandable, testable, and maintainable as the number of agents grows.